Secure Coding Review Report

Title: Secure Coding Review of a Python Flask Login Application

Objective

The purpose of this task is to perform a secure coding review on a Python-based login application to identify security vulnerabilities, analyze risks, and suggest remediation techniques.

Application Details

- Application Type: Flask Login System
- Programming Language: Python
- Framework: Flask
- Database: SQLite

Scope of Review

The review focuses on:

- Authentication mechanism
- Database queries
- Password handling
- Input validation
- Secure coding practices

Tools and Methods Used

Tools Used

Tool	Purpose
Bandit	Static security analysis for Python
Manual Inspection	Detect logical and coding vulnerabilities
VS Code	Source code review

Application Source Code

Vulnerable Login Code

```
from flask import Flask, request
import sqlite3

app = Flask(__name__)

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"

    cursor.execute(query)
    result = cursor.fetchone()

    if result:
        return "Login Successful"
    else:
        return "Invalid Credentials"
```

Static Analysis Using Bandit

Installing Bandit
`pip install bandit`

Running Bandit
`bandit -r project_folder/`

Practical Implementation Steps

Step 1: Install Python

1. Download Python from the official website.
2. Install Python and enable the “Add Python to PATH” option.
3. Verify installation using:

`python –version`

Step 2: Install Flask

`pip install flask`

Step 3: Install Bandit

`pip install bandit`

Step 4: Create Python Project Folder

Create a folder named:
`secure_login_project`
Save the vulnerable login code inside `app.py`.

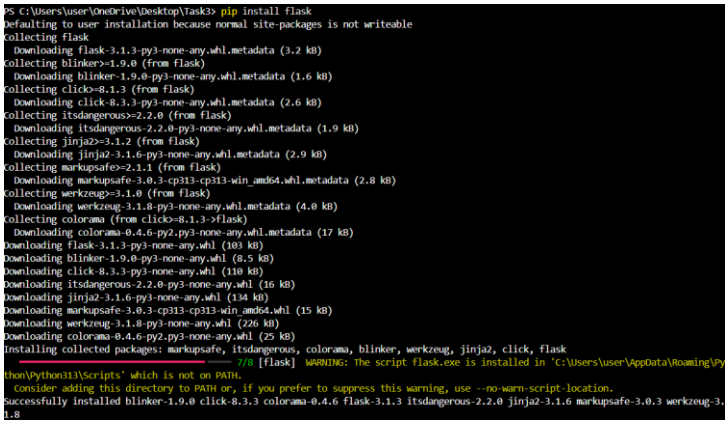
Step 5: Run Application

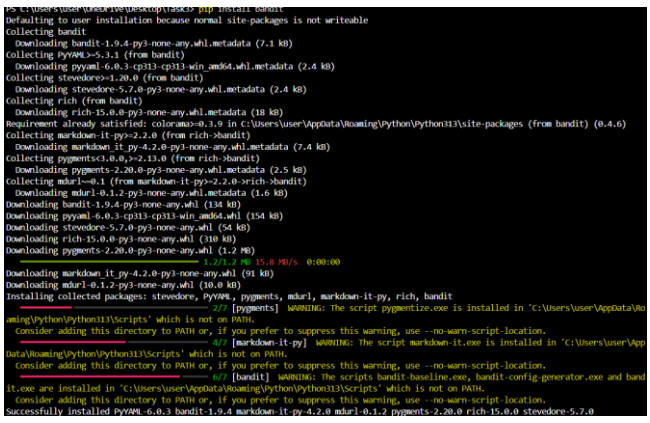
`python app.py`

Step 6: Run Security Scan

`bandit -r .`

Screenshot 1 – Installing flask & bandit





Screenshot 2 – Running Bandit Scan

```
PS C:\Users\user\OneDrive\Desktop\Task3> python -m bandit --version
__main__.py 1.9.4
  python version = 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)]
PS C:\Users\user\OneDrive\Desktop\Task3> python -m bandit -r .
[main]  INFO    profile include tests: None
[main]  INFO    profile exclude tests: None
[main]  INFO    cli include tests: None
[main]  INFO    cli exclude tests: None
[main]  INFO    running on Python 3.13.7
Run started:2026-05-16 07:47:25.910079+00:00

Test results:
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Low
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: .\app.py:14:12
13
14          query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
15

--------------------------------------------------

Code scanned:
        Total lines of code: 16
        Total lines skipped (#nosec): 0

Run metrics:
        Total issues (by severity):
                Undefined: 0
                Low: 0
                Medium: 1
                High: 0
        Total issues (by confidence):
                Undefined: 0
                Low: 1
                Medium: 0
                High: 0
Files skipped (0):
PS C:\Users\user\OneDrive\Desktop\Task3>
```

Vulnerabilities Identified

Vulnerability 1: SQL Injection

Vulnerable Code

```
query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"
```

Description

The application directly concatenates user input into SQL queries. This allows attackers to manipulate database queries.

```python
from flask import Flask, request
import sqlite3

app = Flask(__name__)

Tabnine | Edit | Test | Explain | Document
@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE username='" + username + "' AND password='" + password + "'"

    cursor.execute(query)
    result = cursor.fetchone()

    if result:
        return "Login Successful"
    else:
        return "Invalid Credentials"
```

Risk Level: High

Impact

- Unauthorized login
- Database manipulation
- Data leakage

Example Attack

```
' OR '1'='1
```

This input may bypass authentication.

Remediation

Secure Code

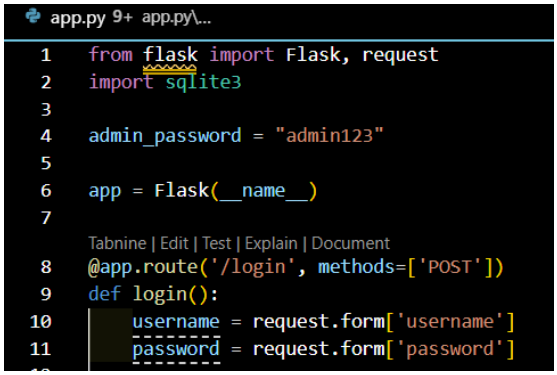
```
query = "SELECT * FROM users WHERE username=? AND password=?"
cursor.execute(query, (username, password))
```

Explanation

Parameterized queries prevent SQL injection by separating user input from SQL commands.

```python
from flask import Flask, request
import sqlite3

app = Flask(__name__)

Tabnine | Edit | Test | Explain | Document
@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = "SELECT * FROM users WHERE username=? AND password=?"
    cursor.execute(query, (username, password))


    cursor.execute(query)
    result = cursor.fetchone()

    if result:
        return "Login Successful"
    else:
        return "Invalid Credentials"
```

Vulnerability 2: Hardcoded Credentials



Vulnerable Code

admin_password = "admin123"

Description

Sensitive credentials stored directly inside source code can be exposed if attackers access the application.

Risk Level: Medium

Impact

- Credential theft
- Unauthorized access

Remediation

Secure Approach

Use environment variables.

```
import os
admin_password = os.getenv("ADMIN_PASSWORD")
```

Vulnerability 3: Plain Text Password Storage

Vulnerable Code

`password = request.form['password']`

Passwords are compared directly without hashing.

Risk Level: High

Impact

- Password theft
- Account compromise

Remediation

Secure Password Hashing

```
from werkzeug.security import generate_password_hash
from werkzeug.security import check_password_hash

hashed_password = generate_password_hash(password)
```

Manual Code Review Findings

Issue	Severity	Recommendation
SQL Injection	High	Use parameterized queries
Hardcoded Password	Medium	Use environment variables
Plain Text Password	High	Use password hashing
Missing Input Validation	Medium	Validate user inputs
No Error Handling	Low	Implement exception handling

Secure Coding Best Practices

Recommended Practices

1. Use parameterized SQL queries
2. Encrypt or hash passwords
3. Avoid hardcoded secrets
4. Validate all user inputs
5. Use HTTPS for secure communication
6. Keep dependencies updated
7. Implement proper authentication
8. Use secure session management
9. Apply least privilege principle
10. Perform regular security testing

8. Improved Secure Login Example

```
from flask import Flask, request
from werkzeug.security import check_password_hash
import sqlite3

app = Flask(__name__)

@app.route('/login', methods=['POST'])
def login():
    username = request.form['username']
    password = request.form['password']

    conn = sqlite3.connect('users.db')
    cursor = conn.cursor()

    query = "SELECT password FROM users WHERE username=?"
    cursor.execute(query, (username,))

    result = cursor.fetchone()

    if result and check_password_hash(result[0], password):
        return "Login Successful"
    else:
        return "Invalid Credentials"
```

Screenshot – Final Secure Login Code

(The final secure login implementation uses parameterized SQL queries and password hash verification to prevent SQL Injection and protect user credentials from exposure.)

Conclusion

The secure coding review identified multiple security vulnerabilities in the Flask login application. Major vulnerabilities included SQL Injection, hardcoded credentials, and insecure password handling.
By implementing parameterized queries, password hashing, environment variables, and proper validation techniques, the application security can be significantly improved.
The review demonstrates the importance of integrating security practices during software development.

Abstract

This project focuses on performing a secure coding review on a Python Flask-based login application. The objective of the review was to identify common security vulnerabilities such as SQL Injection, hardcoded credentials, and insecure password handling. Static analysis tools and manual inspection methods were used to analyze the source code. Appropriate remediation strategies and secure coding practices were also implemented to improve application security.